

openpi 스크립트 정리 최종

π 0 (pi0_base + LoRA) — SO-101 Fine-tuning → Lelsaac Inference → Real Robot

로컬 PC([minsoo-ETRI](#) , Ubuntu 24.04, RTX 3090 ×1) 기준.

전체 구조는 **환경 2개**로 나뉜다.

```
env_isaac (~/IsaacLab/env_isaac)      openpi (~/openpi/.venv)
  Isaac Sim 5.1.0                      JAX
  IsaacLab v2.3.2                      pi0_base + LoRA
  leisaac (task / teleop / eval)       serve_policy.py
      |                               |
      |----- websocket :8000 -----|
```

둘은 websocket으로만 통신한다. 의존성(torch/CUDA vs JAX)이 충돌하므로 **절대 한 venv에 합치지 않는다**.

0. 사전 확인

```
python3.11 --version
nvidia-smi
uv --version
```

필요 시:

```
sudo apt-get update
sudo apt-get install -y ffmpeg libsm6 libxext6 git-lfs
curl -LsSf https://astral.sh/uv/install.sh | sh
```

conda는 사용하지 않는다. 모든 환경은 `uv venv` 또는 `python venv`로 만든다.

목표 조합:

```
Python 3.11
CUDA 12.8
torch 2.7.0+cu128
```

```
Isaac Sim 5.1.0
IsaacLab v2.3.2
```

1. IsaacLab 설치

1-1. clone

공식 IsaacLab을 홈에 직접 clone 한다. **leisaac**을 먼저 받는 것이 아니다.

```
cd ~
git clone https://github.com/isaac-sim/IsaacLab.git
cd IsaacLab
git checkout v2.3.2
```

현재 로컬 상태:

```
HEAD = 37ddf6268 "Bumps version to v2.3.2"
git describe → v2.3.2 (detached HEAD)
```

1-2. venv 생성

venv를 IsaacLab 저장소 **안에** 만든다.

```
uv venv --python 3.11 ~/IsaacLab/env_isaacclab
source ~/IsaacLab/env_isaacclab/bin/activate
```

1-3. torch → Isaac Sim → IsaacLab 순서로 설치

순서를 바꾸면 안 된다. IsaacLab 설치 스크립트가 이미 깔린 torch/isaacsim을 전제로 동작한다.

```
pip install --upgrade pip setuptools wheel

pip install -U torch==2.7.0 torchvision==0.22.0 \
  --index-url https://download.pytorch.org/whl/cu128

pip install "isaacsim[all,extscache]==5.1.0" \
  --extra-index-url https://pypi.nvidia.com
```

```
cd ~/IsaacLab
./isaacclab.sh --install
```

`./isaacclab.sh --install` 이 설치하는 것 (실제 `pip list --editable` 결과):

isaacclab	0.54.2	~/IsaacLab/source/isaacclab
isaacclab_assets	0.2.4	~/IsaacLab/source/isaacclab_asset
s		
isaacclab_contrib	0.0.2	~/IsaacLab/source/isaacclab_contr
ib		
isaacclab_mimic	1.0.16	~/IsaacLab/source/isaacclab_mimic
isaacclab_rl	0.4.7	~/IsaacLab/source/isaacclab_rl
isaacclab_tasks	0.11.12	~/IsaacLab/source/isaacclab_tasks

확인:

```
python -c "import isaacclab, os; print(os.path.dirname(isaacclab.__file__))"
# → /home/minsoo/IsaacLab/source/isaacclab/isaacclab
```

2. Leisaac 설치

2-1. IsaacLab 내부에 clone

leisaac은 `~/IsaacLab/source/` 아래에 clone 한다. IsaacLab의 다른 서브패키지 (`isaacclab_tasks` 등)와 나란히 놓이는 구조다.

```
cd ~/IsaacLab/source
git clone https://github.com/MIMI-MINS00/leisaac-cupstack.git leisaac
```

결과:

```
~/IsaacLab/source/
├─ isaacclab/
├─ isaacclab_assets/
├─ isaacclab_contrib/
```

```
|— isaacsim/
|— isaacsim_rl/
|— isaacsim_tasks/
|— leisaac/          ← 여기
```

현재 HEAD:

```
7e72f12 "Add marker-in-cup task (CupStack)..."
```

submodule에 대하여

leisaac 저장소는 `.gitmodules`에 `dependencies/IsaacLab`을 갖고 있고, 공식 문서는 이 submodule의 IsaacLab을 설치하라고 안내한다.

이 셋업은 그 경로를 따르지 않았다. 1단계에서 설치한 outer `~/IsaacLab`이 실제 설치본이며, `dependencies/IsaacLab`은 venv에 editable로 잡혀 있지 않다.

따라서 `--recursive`는 필요 없고, submodule을 init 하면 IsaacLab이 두 벌이 되어 혼란만 커진다.

2-2. 패키지 설치

leisaac 저장소 안에 다시 `source/leisaac`이 있는 **중첩 구조**다. 설치 경로에 주의.

```
cd ~/IsaacLab/source/leisaac
pip install -e "source/leisaac[lerobot]"
pip install numpy==1.26.0
```

설치 결과:

```
leisaac 0.4.0 ~/IsaacLab/source/leisaac/source/leisaac
```

`[lerobot]` extras는 6단계 데이터 변환에 필요하다.

`numpy==1.26.0`은 반드시 맨 마지막에 실행한다. 앞 단계 패키지들이 numpy를 자동으로 올려버린다.

2-3. 최종 버전 확인

```
pip list | grep -iE "torch|isaac|lerobot|numpy|gymnasium|wa
rp|h5py|opencv"
```

기준값:

```
torch                2.7.0+cu128
torchvision          0.22.0+cu128
torchcodec           0.5
isaacsim (전체)      5.1.0.0
isaacclab            0.54.2
isaacclab_tasks      0.11.12
leisaac              0.4.0
lerobot              0.3.3
numpy                1.26.0
gymnasium            1.2.1
warp-lang            1.15.0
h5py                 3.16.0
opencv-python-headless 4.11.0.86
```

3. Asset 배치

CupStack 태스크에서 사용하는 주요 asset:

```
~/IsaacLab/source/leisaac/assets/
├─ robots/
│   └─ so101_follower.usd
└─ objects/
    ├─ cup.usd
    ├─ marker/marker.usd
    ├─ marker_raw/
    ├─ Marker_3DS.rar
    └─ Marker_.rar
```

Asset	정리
so101_follower.usd	Lelisaac 기본 SO-101 follower asset을 그대로 사용
marker.usd	CupStack용으로 추가한 custom asset. 코드 주석상 스캔 mesh를 실제 보드마커 크기에 맞게 변환·scale 조정
cup.usd	CupStack용으로 추가한 단순 cup asset

CupStack은 별도의 완성형 scene USD를 사용하지 않고, 방/벽/바닥/테이블 등의 geometry를 `CuboidCfg` 등으로 코드에서 직접 구성한다.

환경 변수는 `assets` 디렉터리까지 포함해서 지정한다.

```
export LEISAAC_ASSETS_ROOT=~/.IsaacLab/source/leisaac/assets
```

코드에서는 이 값을 기준으로 예를 들어 다음과 같이 asset 경로를 조합한다.

```
CUP_USD_PATH = Path(ASSETS_ROOT) / "objects" / "cup.usd"
```

4. CupStack 태스크 스펙

4-1. 등록된 환경

source/leisaac/leisaac/tasks/cup_stack/__init__.py

gym id	용도	정의 위치
<code>LeIsaac-S0101-CupStack-v0</code>	teleop 수집 / 정책 평가 (기본)	5행
<code>LeIsaac-S0101-CupStack-Mimic-v0</code>	MimicGen 증강용	14행
<code>LeIsaac-S0101-CupStack-Rewarded-v0</code>	RL(reward shaping)	25행
<code>LeIsaac-S0101-CupStack-Rewarded-State-v0</code>	RL, state-only 관측	37행

env cfg: `cup_stack_env_cfg.py`

CupStackSceneCfg	(162행)	
CupStackStateSceneCfg	(247행)	
CupStackRewardsCfg	(301행)	
CupStackEnvCfg	(384행)	← teleop / 평가용
CupStackRewardedEnvCfg	(434행)	
CupStackStateObservationsCfg	(555행)	
CupStackRewardedStateEnvCfg	(597행)	

4-2. 성공 조건

mdp/terminations.py : `marker_in_cup()`

- | | |
|---------------------|--|
| ① 마커 중심이 컵 입구 반경 이내 | <code>xy_threshold = 0.035 m</code> |
| ② 마커 높이가 컵 안 범위 | <code>z_range = 0.03 ~ 0.09 m (컵 바닥 기준)</code> |

- ③ 마커 기울기 45° 이하 (서 있는 자세)
 ④ require_rest_pose=True 인 경우 팔이 rest pose로 복귀했는지까지 확인

③이 필요한 이유 — 실측 기준값(코드 주석):

상태	dz	기울기
컵 안에 서 있음 (성공)	+0.074	13°
컵 테두리에 놓혀 걸침 (실패)	+0.105	90°
테이블 위 (실패)	+0.009	90°

높이만 보면 "테두리에 걸친 실패"가 성공보다 오히려 높게 나온다. 기울기 조건이 없으면 이 실패가 성공으로 기록된다.

4-3. 관측 / 시물 설정

카메라 2대. `template/single_arm_env_cfg.py` 의 설정은 공식 **Lelsaac v0.4.0 upstream**과 동일하게 그대로 사용하고, CupStack에서는 front 카메라 pose만 task config에서 재정의한다.

```
wrist : {ENV_REGEX_NS}/Robot/gripper/wrist_camera 640×480, FOV≈75°
front  : {ENV_REGEX_NS}/Robot/base/front_camera   640×480, FOV≈78°
둘 다 update_period = 1/30 (30 FPS), PinholeCameraCfg
```

항목	값	비고
<code>episode_length_s</code>	25.0 s	<code>CupStackEnvCfg</code> 기본. RL용 <code>CupStackRewardedEnvCfg</code> 는 50.0 s
<code>decimation</code>	1	<code>single_arm_env_cfg.py:181</code>
sim dt	1/60 s (60 Hz)	leisaac에 오버라이드 없음 → IsaacLab 기본값. 실행 로그 <code>env.yaml</code> 에서 <code>dt: 0.01666...</code> 확인

state / action 은 6차원이다.

```
[shoulder_pan, shoulder_lift, elbow_flex, wrist_flex, wrist_roll, gripper]
= 5-DOF arm + 1 gripper
```

5. Teleoperation 데이터 수집

SO-101 leader 팔로 시물 안의 follower를 직접 조작해 성공 시연을 모은다.

5-1. Leader 포트 확인

LeRobot에서 제공하는 helper로 leader의 serial port를 확인한다.

```
lerobot-find-port
```

케이블을 연결/해제하면서 실제 장치가 `/dev/ttyACM0`, `/dev/ttyACM1` 중 어느 포트인지 확인한 뒤 아래 `--port`에 넣는다.

5-2. 데이터 수집

```
cd ~/IsaacLab/source/leisaac
source ~/IsaacLab/env_isaac/bin/activate
export LEISAAC_ASSETS_ROOT=~/IsaacLab/source/leisaac/assets

python scripts/environments/teleoperation/teleop_se3_agent.py \
    --task=LeIsaac-S0101-CupStack-v0 \
    --teleop_device=so101leader \
    --port=/dev/ttyACM0 \
    --num_envs=1 \
    --device=cuda \
    --enable_cameras \
    --record \
    --dataset_file=./datasets/Marker_97.hdf5
```

옵션	의미
<code>--teleop_device=so101leader</code>	실물 리더암으로 조작. <code>keyboard</code> , <code>gamepad</code> 도 가능
<code>--port</code>	<code>lerobot-find-port</code> 로 확인한 leader serial port
<code>--enable_cameras</code>	front/wrist 이미지 기록. VLA 학습에 필요
<code>--record --dataset_file</code>	성공 시연을 HDF5로 저장
<code>--resume</code>	(선택) 기존 파일에 이어서 기록

SO-101 leader calibration 파일이 없거나 recalibration을 요청하면 teleop 연결 과정에서 calibration이 실행되고, Lelsaac leader 기준 파일은 다음 위치에 저장된다.

```
~/IsaacLab/source/leisaac/source/leisaac/leisaac/devices/le
robot/.cache/so101_leader.json
```


조작 키:

```
b 에피소드 시작
n 성공 종료 → 저장
r 실패 리셋 → 폐기 (파일에 남지 않음)
```

n으로 끝낸 에피소드만 기록되므로, 데이터 품질 게이팅이 수집 단계에서 사람 손으로 이루어진다.

수집 결과 (실제)

~/IsaacLab/source/leisaac/datasets/Marker_97.hdf5

```
episodes          : 97
attrs.total       : 31776
demo_0 keys       : actions, initial_state, obs, processed_actions, states
demo_0.obs keys   : actions, ee_frame_state, front, joint_pos, joint_pos_rel,
                  joint_pos_target, joint_vel, joint_vel_rel, wrist
demo_0.actions    : (410, 6)
```

데이터셋 이름은 이후 `marker_100` 을 사용하지만 실제 수집 성공 에피소드는 **97**개다.

6. LeRobot 포맷 변환

HDF5 → LeRobot Dataset (parquet + mp4).

```
cd ~/IsaacLab/source/leisaac
source ~/IsaacLab/env_isaacclab/bin/activate

python scripts/convert/isaacclab2lerobotv3.py \
  --task_name=LeIsaac-S0101-CupStack-v0 \
  --repo_id=mimiminsoo/marker_100 \
  --hdf5_root=./datasets \
  --hdf5_files=Marker_97.hdf5 \
  --task_description="Pick up the marker and place it into the cup, then reset the arm to rest state." \
  --enable_cameras
```

옵션	의미
<code>--task_name</code>	수집 단계와 동일한 gym id

<code>--repo_id</code>	LeRobot dataset 식별자. 로컬에서는 <code>~/.cache/huggingface/lerobot/<repo_id></code> 에 저장
<code>--hdf5_files</code>	변환할 HDF5 파일. 여러 세션을 묶을 수도 있음
<code>--task_description</code>	학습에 사용되는 language instruction
<code>--push_to_hub</code>	(선택) 변환 후 Hugging Face Hub 업로드

변환 결과 검증

```
cat ~/.cache/huggingface/lerobot/mimiminsoo/marker_100/meta/info.json
```

```
codebase_version : v2.1
total_episodes   : 97
total_frames     : 31291
total_videos     : 194      (97 × 2 카메라)
fps              : 30
action / observation.state : shape 6
observation.images.{front,wrist} : 480×640×3
```

프레임 수 계산

```
HDF5 원본      31776
변환 후        31291
차이           485 = 5 프레임 × 97 에피소드
```

변환 스크립트가 에피소드마다 앞 5프레임을 skip하기 때문에 수치가 정확히 맞는다.

이 파이프라인에서 사용하는 최종 데이터는 **LeRobot v2 계열 구조**이며, openpi는 로컬 cache와 Hugging Face에서 받은 동일 `repo_id` 데이터셋을 같은 형식으로 읽는다.

`meta/` 산출물:

```
episodes.jsonl, episodes_stats.jsonl, info.json, stats.json, tasks.jsonl,
steps_data_index.pkl,
modality.json, stats_gr00t.json ← GR00T 실험용 (π0에는 불필요)
```

7. openpi 설치

```
cd ~
git clone --recurse-submodules https://github.com/EverNorif/openpi.git
cd openpi

GIT_LFS_SKIP_SMUDGE=1 uv sync
GIT_LFS_SKIP_SMUDGE=1 uv pip install -e .
```

항목	의미
<code>--recurse-submodules</code>	<code>third_party</code> 등 서브모듈 포함
<code>GIT_LFS_SKIP_SMUDGE=1</code>	openpi가 lerobot을 git 소스로 받을 때 LFS 대용량 파일 다운로드를 건너뛰는다
<code>uv sync</code>	openpi 전용 venv를 저장소 안에 생성. env_isaacclab 과 절대 공유하지 않는다

현재 HEAD: `285c024 "some fix."`

캐시 위치:

```
~/.cache/openpi/
├─ big_vision/
└─ openpi-assets/      ← pi0_base 등 사전학습 가중치
```

`OPENPI_DATA_HOME` 은 로컬에서는 미설정(기본값 사용). 서버에서만 `/mnt/nvme0` 로 리다이렉트했다.

LoRA는 JAX 구현에만 있다. openpi의 PyTorch 코드베이스는 LoRA를 지원하지 않으므로, 이 파이프라인의 학습은 반드시 JAX 쪽에서 돌아간다.

8. 학습 config 등록

`src/openpi/training/config.py` 에 `TrainConfig` 를 추가한다.

현재 marker 계열 config:

config 이름	데이터	action_horizon	비고
<code>pi0_lora_marker</code>	marker_50	10	초기 실험
<code>pi0_lora_marker_100</code>	marker_100	30	baseline
<code>pi0_lora_marker_100_h10</code>	marker_100	10	horizon 비교용

<code>pi0_150</code>	marker_150	—	서버 학습 checkpoint 서빙용 선언
<code>pi0_180_real</code>	실기 180ep	—	서버 학습 checkpoint 서빙용 선언

`pi0_lora_marker_100` 에서 실제로 중요한 변경점:

항목	$\pi 0$ 기본값	marker_100
초기 weight	random init	<code>pi0_base</code> pretrained checkpoint
action horizon	50	30
PaliGemma	non-LoRA	LoRA rank 16 / alpha 16
Action Expert	non-LoRA	LoRA rank 32 / alpha 32
batch size	32	8 (RTX 3090 24GB 기준)
num workers	2	4
train steps	30,000	30,000 (기본값 유지)
optimizer / LR	AdamW / cosine peak <code>2.5e-5</code>	기본값 유지
dataset	fake/default	<code>mimiminsoo/marker_100</code>
task prompt	사용 안 함	dataset task string 사용

모델의 `action_dim` 자체는 **32로 유지**한다. SO-101의 실제 6D state/action은 `PadStatesAndActions` 에서 32D로 zero-padding되어 $\pi 0$ 에 입력된다.

```

SO-101 6D action/state
      ↓
PadStatesAndActions
      ↓
 $\pi 0$  32D interface

```

SO-101 action 전처리는 arm 5축을 delta action으로, 마지막 gripper 1축을 absolute action으로 사용한다.

```

[shoulder_pan, shoulder_lift, elbow_flex, wrist_flex, wrist_roll] → delta
[gripper]                                                         → absolute

```

핵심 config 형태:

```

Pi0Config(
  paligemma_variant="gemma_2b_lora",
  action_expert_variant="gemma_300m_lora",

```

```

    action_horizon=30,
)

# weight_loader: pi0_base checkpoint
# batch_size: 8
# num_workers: 4
# num_train_steps: 30000
# ema_decay: None

```

`freeze_filter`를 통해 pretrained backbone의 일반 weight는 고정하고 LoRA parameter를 학습한다.

LoRA 사용 목적: 작은 task dataset에 π_0 -base를 효율적으로 적응시키고 반복 실험 비용을 줄이기 위한 fine-tuning 방식이다.

9. norm stats 계산 + Fine-tuning

```

cd ~/openpi
export XLA_PYTHON_CLIENT_MEM_FRACTION=0.9
export HF_HUB_OFFLINE=1

uv run scripts/compute_norm_stats.py --config-name pi0_lora
    _marker_100

uv run scripts/train.py pi0_lora_marker_100 --exp-name=mark
    er100_h30

```

항목	의미
<code>XLA_PYTHON_CLIENT_MEM_FRACTION=0.9</code>	JAX가 GPU 메모리를 90%까지 선점. 기본 0.75로는 batch size가 부족
<code>HF_HUB_OFFLINE=1</code>	데이터셋이 로컬 캐시에만 있을 때 HF Hub 조회를 건너뛴다
<code>compute_norm_stats.py</code>	SO-101 dataset의 state/action 정규화 통계 계산. 데이터셋 또는 관련 전처리 config가 바뀌면 다시 계산
<code>--exp-name</code>	체크포인트 디렉토리 이름
<code>--resume</code>	(선택) 마지막 체크포인트에서 이어 학습

학습 중에는 GPU를 거의 독점하므로, 정책 서버 등 다른 GPU 프로세스를 먼저 종료한다.

산출물

norm stats:

```
~/openpi/assets/pi0_lora_marker_100/mimiminsoo/marker_100/norm_stats.json
+ 각 체크포인트 스텝 디렉토리마다 사본이 함께 저장됨
```

체크포인트:

```
~/openpi/checkpoints/pi0_lora_marker_100/marker100_h30/
├─ 5000/ 10000/ 15000/ 20000/ 25000/ 29999/
└─ wandb_id.txt
```

체크포인트와 데이터셋에 맞지 않는 norm stats를 사용하면 학습/추론 성능이 크게 떨어질 수 있으므로, dataset/config에 맞는 통계를 사용한다.

10. 정책 서버 기동

시물 평가(11단계)와 실로봇(12단계)이 같은 서버를 공유한다.

```
cd ~/openpi
export XLA_PYTHON_CLIENT_MEM_FRACTION=0.35

uv run scripts/serve_policy.py \
  --port 8000 \
  --default-prompt "Pick up the marker and place it into
the cup, then reset the arm to rest state." \
  policy:checkpoint \
  --policy.config=pi0_lora_marker_100 \
  --policy.dir=checkpoints/pi0_lora_marker_100/marker100_
h30/29999
```

옵션	의미
<code>--policy.config</code>	8단계에서 등록한 config 이름
<code>--policy.dir</code>	체크포인트 스텝 디렉토리까지 지정
<code>MEM_FRACTION=0.35</code>	Isaac Sim이 PhysX 씬을 올릴 메모리를 남겨준다. 실로봇만 쓸 때는 높여도 된다

구조:

```
openpi venv                                env_isaac lab venv
serve_policy.py — websocket :8000 → policy_inference.py
y      (11:sim)                                ↳ real_robot_inference.py (12:real)
```

11. Simulation evaluation

터미널 B (10단계 서버가 떠 있는 상태에서).

```
cd ~/IsaacLab/source/leisaac
source ~/IsaacLab/env_isaac lab/bin/activate
export LEISAAC_ASSETS_ROOT=$(pwd)

python scripts/evaluation/policy_inference.py \
  --task=LeIsaac-S0101-CupStack-v0 \
  --eval_rounds=20 \
  --seed=42 \
  --policy_type=openpi \
  --policy_host=localhost \
  --policy_port=8000 \
  --policy_action_horizon=30 \
  --policy_language_instruction="Pick up the marker and place it into the cup, then reset the arm to rest state." \
  --device=cuda \
  --enable_cameras
```

openpi 사용 시 확인할 인자

`policy_inference.py` 는 처음 GR00T용으로 작성된 뒤 여러 backend가 추가되어, 기본값에 GR00T 설정이 남아 있다.

인자	기본값	이 실험에서 사용	이유
<code>--policy_type</code>	<code>gr00tn1.5</code>	<code>openpi</code>	backend 선택
<code>--policy_port</code>	<code>5555</code>	<code>8000</code>	openpi websocket server port

-- policy_action_horizon	16	30	서버가 예측한 action chunk 중 실제 실행할 step 수
-----------------------------	----	----	---

- -policy_action_horizon 은 모델의 prediction horizon을 바꾸는 값이 아니라 execution/receding horizon이다. pi0_lora_marker_100 은 서버에서 30-step chunk를 예측하며, 평가에서는 그중 10/20/30 step처럼 일부만 실행하고 다시 추론하는 것도 가능하다.

전체 인자 목록

```
--task (default None)
--step_hz (default 60)
--seed (default None)
--episode_length_s (default 60.0)
--eval_rounds (default 0 → timeout 없이 성공/수동리셋까지 무한 실행)
--policy_type (default "gr00tn1.5")
--policy_host (default "localhost")
--policy_port (default 5555)
--policy_timeout_ms (default 15000)
--policy_action_horizon (default 16)
--policy_language_instruction (default None)
--policy_checkpoint_path (default None)
--save_debug_frames (flag) 정책이 본 카메라 관측을 전부 PNG로 덤프
--video_dir (default None) 에피소드마다 front 카메라 MP4 1개 저장
```

- -save_debug_frames 는 한 번의 평가에서 수천 장이 생성되므로 디버깅 목적으로만 쓴다.

성공률 기록용으로는 --video_dir 쪽이 실용적이다.

지원되는 --policy_type : gr00tn1.5, gr00tn1.6, lerobot-<model_type>, openpi

12. Real robot inference

정책 서버(10단계)는 그대로 두고, 관측 소스만 시뮬 → 실물로 바꾼다.

```
cd ~/IsaacLab/source/leisaac
source ~/IsaacLab/env_isaac/activate

python scripts/evaluation/real_robot_inference.py \
  --robot_port=/dev/ttyACM0 \
  --front_camera_index=/dev/v4l/by-path/pci-0000:00:14.0-usb-0:10.2:1.0-video-index0 \
  --wrist_camera_index=/dev/v4l/by-path/pci-0000:00:14.0-
```



```
usb-0:10.2:1.0-video-index1 \
  --policy_host=localhost \
  --policy_port=8000 \
  --language_instruction="Pick up the marker and place it
into the cup, then reset the arm to rest state." \
  --action_horizon=10 \
  --num_episodes=5
```

전체 인자 목록

```
--robot_port          (default "/dev/ttyACM0")
--front_camera_index  (default 2)      숫자 index 또는 경로 모두 허용
--wrist_camera_index  (default 0)
--camera_width        (default 640)
--camera_height       (default 480)
--camera_fps          (default 30)
--policy_host         (default "localhost")
--policy_port         (default 8000)    ← 시뮬용 스크립트와 달리 기본값이 이미 8000
--language_instruction (required)
--action_horizon      (default 10)
--step_hz             (default 30.0)
--episode_length_s    (default 30.0)
--num_episodes        (default 5)
--max_relative_target (default 15.0)
```

실기에서만 다른 점

항목	시뮬	실기	이유
action horizon	30	10	더 자주 재추론해 closed-loop에 가깝게. 튜닝 대상이며 시뮬과 맞출 필요 없음
step_hz	60	30	카메라 fps에 맞춤
안전장치	없음	-- max_relative_target=15.0	정책이 튜는 액션을 내도 한 스텝 이동 폭을 제한

카메라 지정

숫자 index(2, 0)는 재연결·재부팅 시 번호가 밀려 **front/wrist가 조용히 뒤바뀐다**. 에러가 나지 않고 성능만 떨어지므로 원인 파악이 어렵다. **by-path** 경로 사용을 권장한다.

현재 로컬 상태:

```
/dev/v4l/by-path/
pci-0000:00:14.0-usb-0:10.2:1.0-video-index0 → ../../vid
eo2
pci-0000:00:14.0-usb-0:10.2:1.0-video-index1 → ../../vid
eo3
(usbv2-* 는 같은 장치를 가리키는 중복 별칭)
```

[확인 필요]

현재 by-path에 나타나는 것은 **USB 포트 하나(10.2)** 아래의 **video-index0 / index1** 두 노드뿐이다.

이는 카메라 2대가 아니라 **UVC 카메라 1대**가 노출하는 두 개의 **video 노드**(capture + metadata)일 가능성이 높다.

실기 실행 시점에는 카메라 2대가 서로 다른 USB 포트에 물려 있었을 것이므로, **실제 실기 실행 시 사용한 by-path 값 2개를 다시 확인해야 한다.**

에피소드 리셋 절차

에피소드 종료 → 팔의 토크가 풀림 → 사람이 손으로 초기 자세로 되돌림
→ Enter → 그 자리에서 토크 재체결 → 다음 에피소드 시작

캘리브레이션

실로봇 follower는 LeRobot의 **S0101Follower**를 사용하며 calibration cache는 LeRobot 쪽 경로를 사용한다.

```
~/.cache/huggingface/lerobot/calibration/robots/s0101_follower/
```

실행 시 해당 robot id에 맞는 calibration이 없으면 LeRobot calibration flow를 거치게 된다. 발표에서는 **leader teleop calibration**과 **real follower calibration**이 서로 다른 경로를 사용한다는 정도만 구분하면 된다.

단계 간 반드시 확인할 항목

아래 항목은 pipeline 재현 시 우선 확인한다.

항목	연결되는 단계	확인 방법
----	---------	-------

model / execution horizon	8 ↔ 11/12	모델 prediction horizon은 checkpoint config에 고정. <code>policy_action_horizon</code> / <code>action_horizon</code> 은 실제 실행 step 수이므로 더 작게 설정 가능
language instruction	6 ↔ 10 ↔ 11 ↔ 12	재현 실험에서는 같은 task instruction을 사용해 조건을 통일
norm stats	6 ↔ 9 ↔ 10	현재 dataset/config에 맞는 <code>norm_stats.json</code> 을 checkpoint와 함께 사용
policy server port	10 ↔ 11/12	openpi server/client 모두 <code>8000</code> 으로 맞춤

최종 실행 순서

```

1. IsaacLab clone (v2.3.2) → ~/IsaacLab
   ↓
2. uv venv → ~/IsaacLab/env_isaacsim
   ↓
3. torch 2.7.0+cu128 → isaacsim 5.1.0 → ./isaacsim.sh --install
   ↓
4. leisaac clone → ~/IsaacLab/source/leisaac (submodule 사용 안 함)
   ↓
5. pip install -e "source/leisaac[lerobot]" → numpy 1.26.0 고정
   ↓
6. asset 확인 (sol101_follower / cup / marker) + LEISAAC_ASSETS_ROOT=.../leisaac/assets
   ↓
7. teleop_se3_agent.py → Marker_97.hdf5 (97 episodes / 31776 frames)
   ↓
8. isaacsim2lerobotv3.py → mimiminsoo/marker_100 (97ep / 31291f / 30fps)
   ↓
9. openpi clone + uv sync
   ↓
10. config.py 에 pi0_lora_marker_100 등록 (H=30, VLM LoRA r16, Action Expert LoRA r32, batch 8)
   ↓
11. compute_norm_stats.py
   ↓
12. train.py → checkpoints/.../marker100_h30/29999
   ↓
13. serve_policy.py (:8000)
   ↓
14. policy_inference.py → CupStack simulation evaluation
   ↓
15. real_robot_inference.py → 실물 S0-101 배포

```

확인 필요 목록

현재 파이프라인의 핵심 동작은 대부분 확인됐다. 재현 전에 실제 장치 상태만 다시 확인하면 된다.

#	항목	내용
1	Leader serial port	<code>lerobot-find-port</code> 로 현재 <code>/dev/ttyACM*</code> 번호 확인
2	실기 follower port	real inference 실행 전 현재 follower port 확인
3	실기 카메라 경로	front/wrist가 실제로 연결된 <code>/dev/v4l/by-path/...</code> 두 경로 재확인
4	calibration	현재 장치에 대응하는 leader/follower calibration 파일이 존재하는지 확인
5	Hugging Face 업로드	재현 시 local-only로 쓰지 <code>--push_to_hub</code> 로 Hub까지 올릴지 선택